

APPCRONS WEB

Product Specification Document

End-to-End (E2E) Test Specification for TestSprite

Document Version	1.0
Status	Final — Ready for TestSprite Execution
Application	Appcrons Web (Next.js App Router)
Repository	https://github.com/Tibz-Dankan/appcrons-web
Framework	Next.js 14 · TypeScript · Redux · React Query
Prepared For	TestSprite E2E Automation
Total Test Cases	12
Date	April 16, 2026

1. Document Overview

This Product Specification Document defines the end-to-end (E2E) test scenarios for the Appcrons Web platform. It is structured for direct consumption by TestSprite — an AI-driven E2E test execution engine — and describes 12 complete user workflows with exact navigation paths, UI elements, step-by-step actions, and expected outcomes.

1.1 Purpose

The purpose of this document is to provide TestSprite with a precise, unambiguous specification from which to generate, configure, and execute E2E tests against the Appcrons web frontend.

1.2 Application Summary

Appcrons is a SaaS platform that optimises the uptime of free-tier backend instances (e.g. on Render) by making periodic HTTP requests at defined intervals. The web frontend is built with Next.js App Router, TypeScript, Redux Toolkit for global state management, and React Query for server-state management.

1.3 Page / Route Map

The following table enumerates every Next.js App Router route used across the 12 test cases:

Route	Access	Purpose
/auth/signup	Public	New user registration
/auth/login	Public	Returning user login
/auth/forgot-password	Public	Initiate password reset
/dashboard	Protected	Lists all user applications
/dashboard?action=createapp&step;=1	Protected	Step 1 — Create app form (modal)
/dashboard?action=enableapp&step;=2&appId;={id}	Protected	Step 2 — Enable app (modal)
/dashboard?action=finish&step;=3&appId;={id}	Protected	Step 3 — Finish creation (modal)
/app/{appId}	Protected	Application details, RTF, update, delete
/settings	Protected	Personal details & password change
/	Public	Home page (theme toggle accessible here)

2. Test Conventions & Data Strategy

2.1 Test Data Generation

IMPORTANT: Each test run must use freshly generated test data. Credentials (email, username) must be unique per run. TestSprite should use dynamic data generation (e.g. timestamps or random suffixes) for email and username fields to avoid conflicts with existing accounts.

- Email: testuser_{timestamp}@appcrons-test.com
- Username: testuser_{timestamp}
- Password: Test@1234 (satisfies 5–30 char requirement)
- App Name: TestApp_{timestamp}
- App URL: https://testapp-{timestamp}.onrender.com/active

2.2 Test Dependency Chain

Test Case	Title	Depends On
TC-01	Signup	None — standalone
TC-02	Login	TC-01 (Signup) then Logout
TC-03	Create Application	TC-01 (Signup → Dashboard)
TC-04	Update Application	TC-01 + TC-03 (on /app/{appld})
TC-05	Delete Application	TC-01 + TC-03 (on /app/{appld})
TC-06	Disable from Dashboard	TC-01 + TC-03 (enabled app on /dashboard)
TC-07	Disable from Details	TC-01 + TC-03 (enabled app on /app/{appld})
TC-08	Add RTF	TC-01 + TC-03 (on /app/{appld})
TC-09	Delete RTF	TC-01 + TC-03 + TC-08 (RTF exists)
TC-10	Update Personal Details	TC-01 → /settings
TC-11	Change Password	TC-01 → /settings
TC-12	Toggle Theme	None — no login required

2.3 Authentication Handling

TestSprite should store the authenticated session (cookies/JWT) after a successful signup or login and reuse it across dependent steps within the same test case. Each top-level test case should begin with a fresh signup using unique credentials to ensure test isolation.

2.4 Timing & Async Behaviour

- After clicking 'Create' on the signup form, allow up to 10 seconds for the redirect to /dashboard (the app shows a 5-second spinner then redirects).
- After application creation (TC-03 Step 7), the modal auto-advances to Step 2 upon API success — wait for URL to update.

- After application deletion (TC-05), the app shows a success toast, then an info toast, then redirects to /dashboard after ~7 seconds total.
- All toggle (enable/disable) operations should wait for the spinner to disappear before asserting state.

3. E2E Test Case Specifications

Each test case below is self-contained and includes the full setup flow (pre-conditions), ordered test steps, expected results per step, page URLs, and UI elements targeted.

TC-01 User Signup		
Step #	Test Action	Expected Result
1	Navigate to /auth/signup	Signup page loads with form and Appcrons logo visible
2	Enter a valid unique username in the 'Username' field	Username value accepted and reflected in the field
3	Enter a valid unique email address in the 'Email' field	Email value accepted and reflected
4	Enter a valid password (5–30 chars) in the 'Password' field	Password is masked and accepted
5	Click the 'Create' button	Button shows 'Creating' spinner; form disabled during API call
6	Wait for spinner ('Creating...' then 'Redirecting...')	Success notification card appears; 'Redirecting' spinner shown
7	Observe page redirect	User is redirected to /dashboard within ~5 seconds of signup success
Objective	Verify that a new user can successfully register an account on Appcrons.	
Pre-conditions	Application is running. No existing account with the test email. Test data: unique username, valid email, password (min 5, max 30 chars).	
Page URL	/auth/signup	
UI Elements	Username input, Email input, Password input, Create button, 'Already have an account? Log in' link, Appcrons logo	

TC-02 User Login (after Signup and Logout)		
Step #	Test Action	Expected Result
1	Complete TC-01 (Signup) and land on /dashboard	Dashboard is accessible; user is authenticated
2	Open the user dropdown in the dashboard header	Dropdown opens showing the logout option
3	Click 'Log out' in the dropdown	Logout action is triggered
4	Confirm logout and redirect to /auth/login	User session cleared; user lands on /auth/login
5	Enter the same email used during signup	Email field accepts and reflects the value
6	Enter the same password used during signup	Password field is masked and accepts the value

7	Click the 'Log in' button	'Logging in' spinner appears; button disabled during API call
8	Observe spinner and redirect	User is authenticated and redirected to /dashboard
Objective	Verify that an existing user can log in using credentials created during signup, after logging out.	
Pre-conditions	A registered account exists (TC-01). User is authenticated from signup and on the dashboard.	
Page URL	Signup: /auth/signup → Dashboard: /dashboard → Login: /auth/login	
UI Elements	NavDropDown logout option, 'Log out' button with LogoutIcon, Login form (Email, Password), 'Log in' button, 'Forgot password?' link, 'Create account' link	

TC-03 Create / Add Application

Step #	Test Action	Expected Result
1	Sign up (TC-01) and land on /dashboard	Dashboard loads; user is authenticated
2	Click the 'New' button (PlusIcon) to open the modal	Modal opens at Step 1 (?action=createapp&step;=1)
3	Observe the 3-step progress panel	Progress panel shows 3 steps with Step 1 active
4	Fill in 'Application Name' with a valid value	Name field accepts input
5	Fill in 'URL Endpoint' with a valid URL	URL field accepts a valid URL
6	Select a 'Request Interval' (5, 10, or 15 minutes)	Interval dropdown shows options 5, 10, 15; one selected
7	Click 'Submit'	'Submitting' spinner appears; button disabled; API call made
8	Wait for success notification; modal advances to Step 2	Success toast shown; URL updates to ?action=enableapp&step;=2&appId;={id}
9	Toggle the enable switch ON for the application	Toggle switches to enabled (green); success toast shown
10	Wait for success; modal advances to Step 3	URL updates to ?action=finish&step;=3&appId;={id}
11	Observe Finish step and click navigate action	Modal closes; user navigated to /app/{appId}
Objective	Verify the full 3-step flow for creating a new application: form submission, enabling the app, and navigating to the app details page.	
Pre-conditions	User is signed up and on /dashboard. Test data ready: app name, valid URL endpoint, request interval.	
Page URL	/dashboard (modal) → ?action=createapp&step;=1 → ?action=enableapp&step;=2&appId;={id} → ?action=finish&step;=3&appId;={id} → /app/{appId}	
UI Elements	'New' button (PlusIcon) in dashboard header, PostApp form (Application Name, URL Endpoint, Request Interval select), Submit button, ToggleSwitch (enable), Skip button, PostAppProgress sidebar	

TC-04 Update Application Info

Step #	Test Action	Expected Result
--------	-------------	-----------------

1	Sign up (TC-01) and create an application (TC-03) to land on /app/{appId}	App details page loads with current data pre-populated in update form
2	Locate the 'Update Application Information' card/form	Update form is visible with pre-populated current values
3	Clear and update the 'Application Name' field	New name is entered
4	Clear and update the 'URL Endpoint' field	New URL is entered
5	Change the 'Request Interval' dropdown	New interval is selected
6	Click 'Save Changes'	'Saving' spinner appears; button disabled during API call
7	Observe spinner and response	Success notification; form reflects updated values from server

Objective	Verify that a user can update the name, URL endpoint, and/or request interval of an existing application from the app details page.
Pre-conditions	User is signed up, an application exists (TC-03). User is on /app/{appId}.
Page URL	Signup → /dashboard → Create app (TC-03) → /app/{appId}
UI Elements	UpdateAppCard component, 'Update Application Information' form, Application Name input, URL Endpoint input, Request Interval select, 'Save Changes' button

TC-05 Delete Application

Step #	Test Action	Expected Result
1	Sign up (TC-01) and create an application (TC-03)	App details page loads; Delete section is visible
2	Scroll to the 'Delete this Application' section	'Delete this Application' section shows with 'Delete app' button
3	Click the 'Delete app' button (red) to open the modal	Confirmation modal opens with explanation and typed-input requirement
4	Read the confirmation instruction	Instruction shows app name in red
5	Type exactly 'delete application {appName}' in the input	Input accepts text; validation runs per keystroke
6	Observe the Delete button state	Delete button becomes enabled only when input exactly matches
7	Click the 'Delete' button	'Deleting' spinner appears; button disabled during API call
8	Wait for spinner and success notification	Success + 'Redirecting to dashboard' info toast appears
9	Observe redirect to dashboard	User redirected to /dashboard; application no longer in list

Objective	Verify that a user can permanently delete an application using the confirmation modal with the required typed confirmation.
Pre-conditions	User is signed up, an application exists (TC-03), and user is on /app/{appId}.
Page URL	Signup → /dashboard → Create app (TC-03) → /app/{appId}

UI Elements	DeleteApp component, 'Delete app' button (red), Modal with typed confirmation input, Cancel button, Delete button (red)
--------------------	---

TC-06 Disable Application from Dashboard

Step #	Test Action	Expected Result
1	Sign up (TC-01) and create an enabled application (TC-03)	Dashboard loads; authenticated session active
2	Navigate to /dashboard	Dashboard shows the application list
3	Locate the created application card	Application card is visible with current enabled state
4	Observe the toggle switch is in 'enabled' state (green)	Toggle is green/active indicating enabled
5	Click the toggle switch to disable the application	Spinner appears during API call; toggle non-interactive
6	Wait for spinner and API response	Toggle switches to grey/off; success notification; app state updated in Redux

Objective	Verify that a user can disable an enabled application directly from the dashboard application list.
Pre-conditions	User is signed up, an enabled application exists (TC-03), and user is on /dashboard.
Page URL	Signup → Create app (TC-03, enabled) → /dashboard
UI Elements	AppList, EnableDisableAppCard/EnableDisableApp toggle switch per app card, ToggleSwitch (green = enabled, grey = disabled)

TC-07 Disable Application from App Details Page

Step #	Test Action	Expected Result
1	Sign up (TC-01) and create an enabled application (TC-03)	App details page loads; app shown in enabled state
2	Locate the enable/disable toggle in EnableDisableAppCard	Enable/Disable card is visible
3	Observe toggle is in 'enabled' state (green)	Toggle clearly reflects enabled state (green/on)
4	Click the toggle to disable the application	Spinner appears; toggle non-interactive during API call
5	Wait for spinner and API response	Toggle switches to grey/off; success notification; global state updated

Objective	Verify that a user can disable an enabled application from the application details page.
Pre-conditions	User is signed up, an enabled application exists (TC-03), and user is on /app/{appId}.
Page URL	Signup → Create app (TC-03, enabled) → /app/{appId}
UI Elements	EnableDisableAppCard on /app/{appId}, EnableDisableApp ToggleSwitch, Spinner overlay during pending state

TC-08 Add Request Time Frame (RTF) for Application

Step #	Test Action	Expected Result
1	Sign up (TC-01) and create an application (TC-03)	App details page loads; Add RTF card visible
2	Locate the 'Add Request Time Frame' card/section	Form shown with timezone selection or existing timezone label
3	If no existing timezone: select timezone from the TimeZone dropdown	Timezone selected and stored in formik state
4	Observe the 'Existing Time Ranges' grid	Existing ranges (if any) displayed in 12-hour format in 2-column grid
5	Select a 'Start time' from the dropdown (e.g. 06:00AM)	Start time dropdown updates formik value
6	Select an 'End time' from the dropdown (e.g. 11:00PM)	End time dropdown updates formik value
7	Observe validation message (green = valid, red = conflict)	Validation: green check if no conflict; red error if overlapping
8	Click 'Submit' when validation shows green	'Processing' spinner appears; Submit button disabled; API call made
9	Wait for spinner and API response	Success notification; new time range in grid; Redux state updated

Objective	Verify that a user can add a new Request Time Frame (start time, end time, and timezone) for an application.
Pre-conditions	User is signed up, an application exists (TC-03), and user is on /app/{appId}.
Page URL	Signup → Create app (TC-03) → /app/{appId}
UI Elements	PostRequestTimeRangeCard, PostRequestTimeRange form, TimeZoneSelect (if no timezone), Existing Time Ranges grid, Start time InputSelect, End time InputSelect, Submit button, Validation message (CheckIcon / ErrorIcon)

TC-09 Delete Application RTF

Step #	Test Action	Expected Result
1	Sign up (TC-01), create app (TC-03), add RTF (TC-08)	App details page loads; RTF section visible with at least one entry
2	Scroll to the 'Request Time Frames' section	RTF section shows all existing time ranges
3	Locate the RTF entry to delete	Target RTF visible with start and end times in 12-hour format
4	Click the delete action on the RTF item	Confirmation form opens showing RTF times to be deleted
5	Read the confirmation message	Confirmation text identifies the RTF by time range
6	Click the 'Delete' button (red) to confirm deletion	'Deleting' spinner appears; Delete button disabled during API call

7	Wait for spinner and API response	Success notification; RTF removed from list; Redux state updated; section hides if no more RTFs
Objective	Verify that a user can delete an existing Request Time Frame from an application.	
Pre-conditions	User is signed up, an application with at least one RTF exists (TC-08 completed), and user is on /app/{appId}.	
Page URL	Signup → Create app (TC-03) → Add RTF (TC-08) → /app/{appId}	
UI Elements	RequestTimeRangeCard, RTF item with delete action, DeleteRequestTimeRange confirmation form, Cancel button, Delete button (red #D9534F), Spinner	

TC-10 Update Personal Details in Settings

Step #	Test Action	Expected Result
1	Sign up (TC-01) and land on /dashboard	Dashboard loads; user authenticated
2	Navigate to /settings	Settings page (/settings) loads with SettingsPageLayout wrapper
3	Locate the 'Personal Details' form	UpdatePersonalDetails form is visible
4	Observe fields pre-populated with current user data	Fields pre-populated with user.name and user.email from Redux data
5	Update the 'Username' field with a new value	New username value entered
6	Update the 'Email' field with a new valid email	New email value entered
7	Click 'Save Changes'	'Saving' spinner appears; button disabled during API call
8	Wait for spinner and API response	Success notification; Redux user state updated with new values
Objective	Verify that a logged-in user can update their username and email address from the Settings page.	
Pre-conditions	User is signed up and authenticated.	
Page URL	Signup → /settings (via navigation)	
UI Elements	SettingsPageLayout, UpdatePersonalDetails form, Username (name) input, Email input, 'Save Changes' button, Spinner	

TC-11 Change Password in Settings

Step #	Test Action	Expected Result
1	Sign up (TC-01) and land on /dashboard	Dashboard loads; user authenticated
2	Navigate to /settings	Settings page loads; Change Password form visible
3	Locate the 'Change Password' form	Change Password section visible with two masked password fields
4	Enter the current password in 'Current Password' field	Current password masked and accepted

5	Enter a new valid password (5–30 chars) in 'New Password' field	New password masked, accepted, meets validation (5–30 chars)
6	Click 'Submit'	'Processing' spinner appears; Submit disabled during API call
7	Wait for spinner and API response	Success notification shown; fields reset to empty; user stays on settings page

Objective	Verify that a logged-in user can change their account password from the Settings page.
Pre-conditions	User is signed up and authenticated. User knows their current password.
Page URL	Signup → /settings
UI Elements	SettingsPageLayout, ChangePassword form, Current Password input (masked), New Password input (masked), Submit button, Spinner

TC-12 Toggle Dark / Light Theme

Step #	Test Action	Expected Result
1	Navigate to / (home page)	Page loads in current theme (light or dark)
2	Locate the theme toggle switch in the navigation bar	Theme toggle visible in navbar with appropriate icon
3	Observe current theme state	Toggle reflects current theme state
4	Click toggle to switch to dark mode	Toggle switches to dark mode; dark background applied; colorScheme set to 'dark'
5	Observe page appearance change	All page elements reflect dark theme
6	Click toggle again to switch back to light mode	Toggle switches back to light mode; light background applied
7	Observe page appearance change	All page elements reflect light theme; preference persisted via next-themes

Objective	Verify that the theme toggle switch correctly switches the application between light and dark modes without requiring authentication.
Pre-conditions	Application is accessible (any page). No login required.
Page URL	/ (home page) or any page with ThemeController in the header
UI Elements	ThemeController component, ToggleSwitch (MoonIcon = light, SunIcon = dark), next-themes setTheme()

4. Global Assertions & Validation Rules

4.1 Form Validation Rules (Client-Side — Yup)

Field	Rule	Expected Behaviour
Signup: Username	Required, max 255 chars	Error shown if empty or >255 chars
Signup: Email	Required, valid email	Error shown if empty or invalid email
Signup/Login: Password	Required, min 5, max 30 chars	Error shown if too short, too long, or empty
Login: Email	Required	Error shown if empty
App: Name	Required, max 255 chars	Error if empty
App: URL	Required, max 255 chars	Error if empty
App: Request Interval	Required; options: 5/10/15	One of 3 dropdown options must be selected
RTF: Start Time	Required	Error if empty; cross-field validation vs end time
RTF: End Time	Required	Error if empty; must not conflict with existing ranges
Settings: Username	Required, min 5, max 30 chars	Error if too short or empty
Settings: Email	Required, max 255 chars	Error if empty or invalid
Change Pwd: Current	Required	Error if empty
Change Pwd: New	Required, min 5, max 30 chars	Error if too short, too long, or empty
Delete App: Confirm	Exact match: 'delete application {name}'	Delete button disabled until exact match typed

4.2 Notification / Toast Assertions

- A notification card appears within 3 seconds of the API response.
- Notification type matches outcome: 'success' (green), 'error' (red), or 'info' (blue).
- Notification auto-dismisses after 5 seconds (some info toasts after 3 seconds).

4.3 Toggle Switch Assertions

- Assert the initial state before interaction.
- Assert a spinner/loading overlay appears during the API call (prevents double-click).
- Assert the toggle state changes to the expected value after API success.
- Assert the toggle reverts if the API call fails.

5. Glossary

Term	Definition
Appcrons	The SaaS platform that sends periodic HTTP pings to keep free-tier backends alive.
RTF	Request Time Frame — a time range (start–end + timezone) during which Appcrons sends requests.
TestSprite	The AI-driven E2E test execution tool consuming this specification.
Next.js App Router	File-based routing system in Next.js 13+; each folder with page.tsx defines a route.
Redux Store	Global client-side state; auth, app list, and notifications are stored here.
React Query	Server-state library for API calls; provides isPending, isError, data states.
ToggleSwitch	Custom toggle component used for enable/disable and theme switching.
Modal	Overlay dialog used for multi-step app creation, delete confirmation, and RTF management.
PostAppLayout	The modal component managing the 3-step app creation flow via URL query params.
ThemeController	Component wrapping next-themes to toggle dark/light mode.
isDisabled	Boolean field on the App model; true = app is disabled (not sending requests).
accessToken	JWT stored client-side after authentication; sent with all protected API calls.

— End of Document —